EXPLORATORY DATA ANALYSIS AND VISUALIZATION

Student Academic Success

Student Name	Adharsh R
Roll Number	24AD002
Department	Artificial Intelligence and Data Science
Institution	KPR Institute of Engineering and Technology
Course Code	U21ADP05
Academic Year	2026–2027 (Odd Semester)

Case Study Report based on the executed Google Colab project notebook

Abstract

This project applies exploratory data analysis, statistical visualization, and a Multilayer Perceptron (MLP) classifier to the UCI Predict Students' Dropout and Academic Success dataset. The dataset contains 4,424 student records and 37 fields, including the target outcome. The analysis inspects data quality, explores distributions and feature relationships, and evaluates a three-class model for Dropout, Enrolled, and Graduate outcomes. The executed notebook reports 73.64% test accuracy on a held-out test set of 664 students. Results indicate stronger classification performance for Graduate and Dropout than for Enrolled students. The report includes the actual plots and metrics exported in the submitted notebook PDF.

1. Introduction

Educational data can help institutions understand patterns associated with student progression. This case study explores a public dataset of student demographic, academic, and socioeconomic attributes. The goal is descriptive analysis and demonstration of a reproducible classification workflow; model predictions should not be treated as causal explanations or as a replacement for academic support staff.

Project objectives

• Inspect dataset structure and data quality.

• Explore outcome balance, numerical distributions, correlations, and potential outliers.

• Create and interpret at least five visualizations.

• Train and evaluate an MLP classifier using train, validation, and test splits.

• Summarize results, limitations, and future improvements.

2. Dataset Description

Dataset: Predict Students' Dropout and Academic Success. Source: UCI Machine Learning Repository. The executed notebook loaded 4,424 rows and 37 columns (36 predictor fields plus the Target field). The target contains three classes: Graduate, Dropout, and Enrolled. The notebook reports 2,209 Graduate records, 1,421 Dropout records, and 794 Enrolled records. Dataset reference: https://doi.org/10.24432/C5MC89.

Target class counts

Outcome	Count	Share
Graduate	2,209	49.9%
Dropout	1,421	32.1%
Enrolled	794	17.9%

The target distribution is imbalanced: Graduate is the largest class and Enrolled is the smallest. This matters when interpreting overall accuracy.

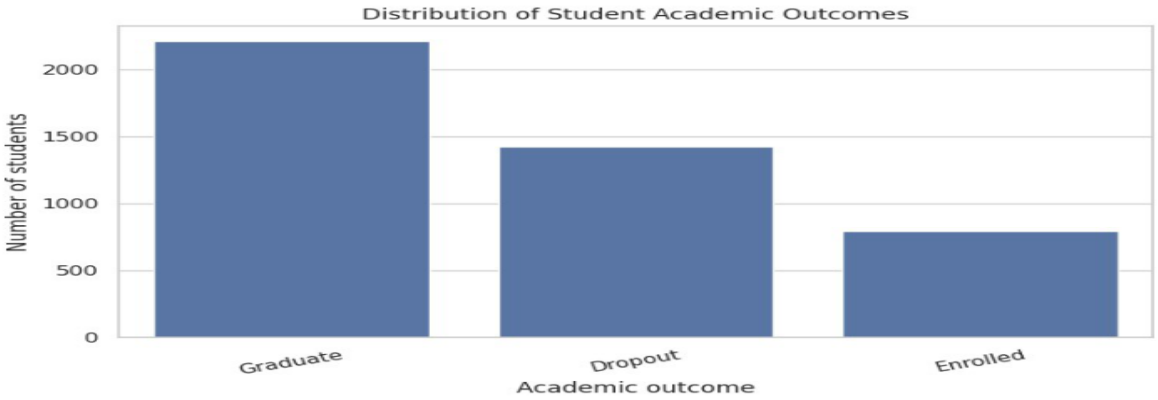
3. Data Understanding and Preprocessing

The notebook reports 29 integer columns, 7 floating-point columns, and one object column before preprocessing. The missing-value inspection shows zero missing values for the displayed fields, and the duplicate-row count is zero. Exact duplicate rows were removed as a safeguard. A preprocessing pipeline was created to impute numerical values with the median and standardize them, and to impute categorical values with the most frequent category and one-hot encode them. The actual detected feature columns were all numeric, so the categorical branch was not used for this dataset run. Data was split into 3,096 training records, 664 validation records, and 664 test records.

4. Exploratory Data Analysis and Visualizations

Figure 1. Student outcome distribution

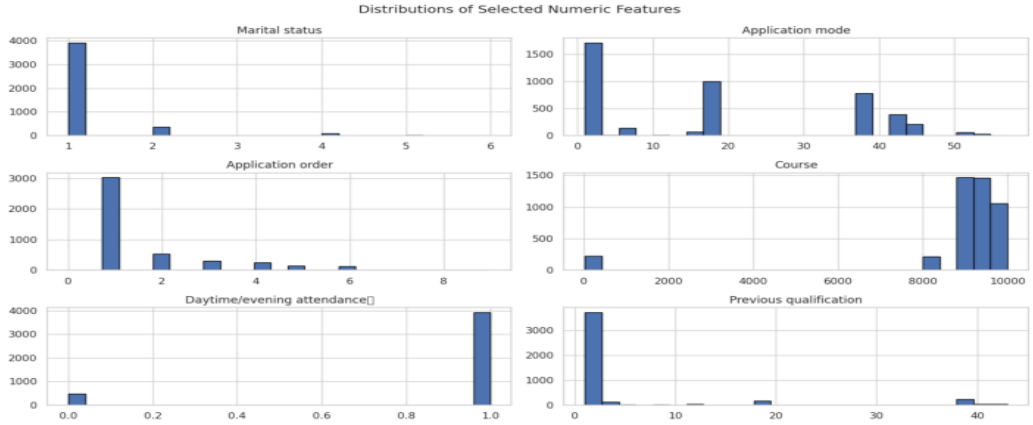
```
# Visualization 1: target class distribution
plt.figure(figsize=(8,5))
order = df[target_col].value_counts().index
sns.countplot(data=df, x=target_col, order=order)
plt.title("Distribution of Student Academic Outcomes")
plt.xlabel("Academic outcome"); plt.ylabel("Number of students")
plt.xticks(rotation=15); plt.tight_layout(); plt.show()
print("Interpretation: Compare the number of students in each outcome
class; note any class imbalance.")
```



Interpretation: Graduate (2,209) is the most frequent outcome, followed by Dropout (1,421) and Enrolled (794). Because the class counts differ, accuracy alone does not fully describe performance across all groups.

Figure 2. Distributions of selected numeric features

```
unusual values in the selected features.")
else:
    print("No numeric columns available for histograms.")
```

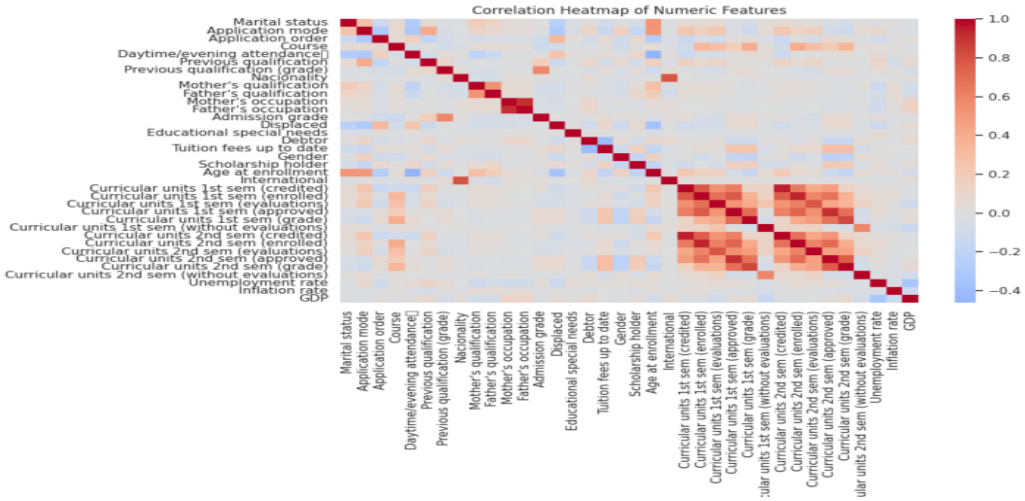



```
Interpretation: Identify skewness, spread, and possible unusual values
in the selected features.
```

Interpretation: The selected variables are discrete-coded fields and have uneven distributions. Several variables are concentrated in a small number of encoded categories. This is a reminder that numeric storage does not necessarily mean a variable is continuous.

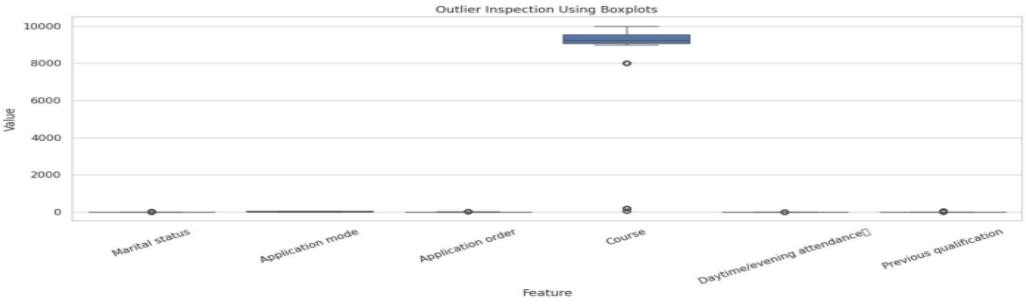
Figure 3. Correlation heatmap

```
else:
    print("Not enough numeric columns for correlation analysis.")
```



Interpretation: The heatmap shows clusters of correlation among some curricular-unit variables. Correlations can identify related features, but they do not establish causal relationships. Because many features are categorical codes, correlation values should be interpreted cautiously.

Figure 4. Boxplots for selected fields

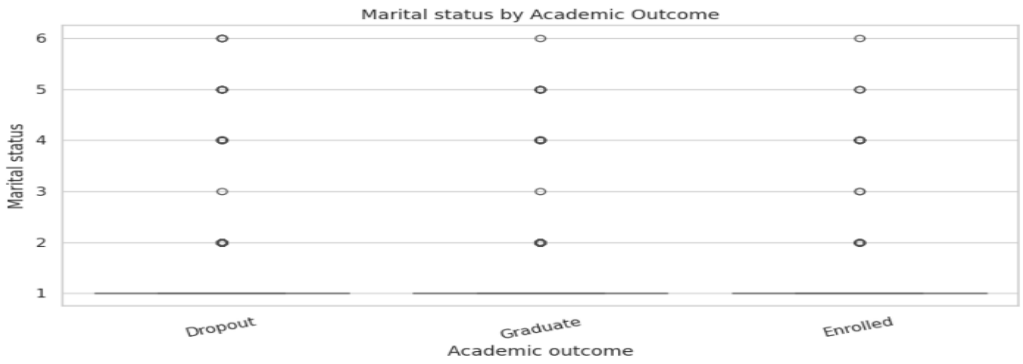



```
Interpretation: Points beyond whiskers may be potential outliers;
investigate before removing them.

# Visualization 5: feature relationship to outcome
# Use a numeric feature with variation; plot outcome-wise
distributions.
if numeric_cols:
    chosen = numeric_cols[0]
```

Interpretation: The fields have different scales. The Course field dominates the combined plot because its numeric codes are much larger than those of the other selected variables. Separate or standardized plots would make outlier analysis more meaningful; apparent outliers should not be removed without context.

Figure 5. Marital status by outcome




```
Interpretation: Compare the distribution of Marital status across
outcome groups; differences are associations, not proof of cause.
```

Interpretation: The plotted variable is a discrete-coded category, not a continuous measurement. A count or proportion plot by outcome would be more interpretable than a boxplot for this feature. This figure is included as the actual notebook output and highlights an improvement opportunity in visualization choice.

5. Deep Learning Model

The project uses a Multilayer Perceptron classifier (MLP) for three-class prediction. Two configurations were compared: hidden layers (64, 32) with alpha 0.0001 and hidden layers (100, 50) with alpha 0.001. The selected configuration was (64, 32), based on the validation comparison. Both configurations reached the configured maximum of 150 iterations, so the training process had not necessarily reached a formal convergence stop before the iteration limit.

6. Model Evaluation

The held-out test set contains 664 records. The reported test accuracy is 0.7364 (73.64%). Class-wise results are shown below.

Class	Precision	Recall	F1-score	Support
Dropout	0.77	0.77	0.77	214
Enrolled	0.41	0.39	0.40	119
Graduate	0.83	0.84	0.83	331
Macro average	0.67	0.67	0.67	664
Weighted average	0.73	0.74	0.73	664

Interpretation: The model performs best on Graduate (F1 = 0.83) and Dropout (F1 = 0.77). Enrolled is harder to identify (F1 = 0.40), with recall of 0.39. The model may confuse Enrolled students with the other outcomes. This is the key performance limitation and should be reported alongside overall accuracy.

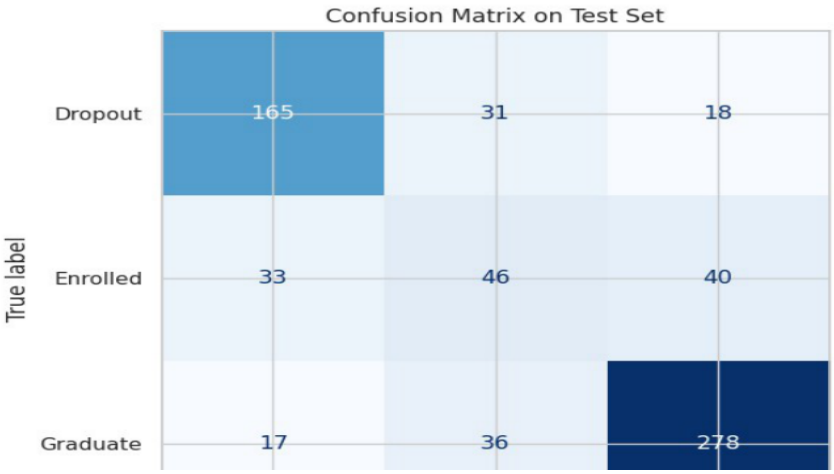
Figure 6. Training loss versus iteration




```
Interpretation: A decreasing loss suggests learning; a plateau may
indicate convergence.
# Evaluation visualization 2: confusion matrix
```

Interpretation: Training loss decreases over the 150 iterations, indicating that the optimization is learning the training data. Since the curve is still declining near the end and the model reached the iteration cap, additional iterations or hyperparameter tuning could be investigated.

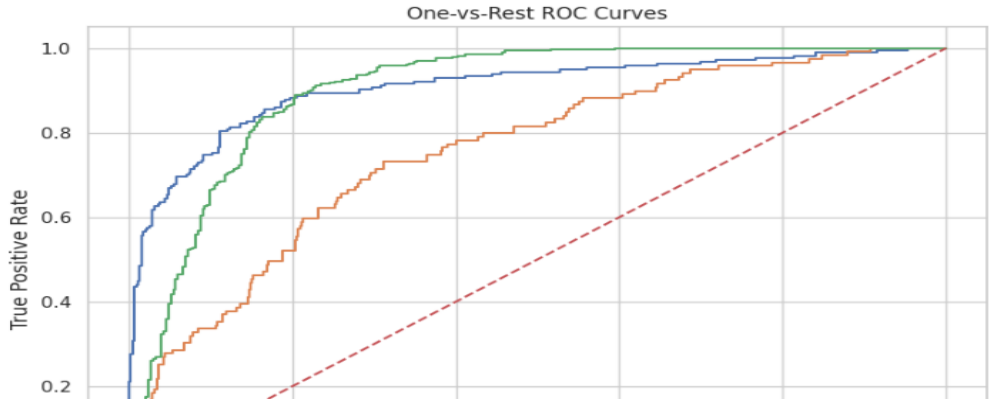
Figure 7. Confusion matrix



Interpretation: The diagonal cells represent correct predictions. The matrix shows 165 correct Dropout predictions, 46 correct Enrolled predictions, and 278 correct Graduate predictions. Enrolled records are often classified as Dropout (33) or Graduate (40), which explains the weaker recall for that class.

Figure 8. One-vs-rest ROC curves

```
    roc_results.append((cls, roc_auc))
    plt.plot(fpr, tpr, label=f"{cls} (AUC={roc_auc:.3f})")
plt.plot([0,1], [0,1], linestyle="--")
plt.xlabel("False Positive Rate"); plt.ylabel("True Positive Rate")
plt.title("One-vs-Rest ROC Curves")
plt.legend(); plt.tight_layout(); plt.show()
print("One-vs-rest AUC:", roc_results)
```



The notebook reports AUC values of 0.907 for Dropout, 0.755 for Enrolled, and 0.901 for Graduate. The ranking performance is strongest for Dropout and Graduate; Enrolled remains more difficult to distinguish.

Figure 9. Per-class precision, recall, and F1-score

```
ylim=(0,1))
plt.title("Per-Class Evaluation Metrics")
plt.ylabel("Score"); plt.xticks(rotation=15); plt.tight_layout();
plt.show()
```


```
{"summary":"{\n  \"name\": \"metrics_df\",\n  \"rows\": 3,\n
\"fields\": [\n    {\n      \"column\": \"Class\",\n
\"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 3,\n        \"samples\": [\n
\"Dropout\",\n          \"Enrolled\",\n          \"Graduate\"\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"Precision\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.22734218279640295,\n        \"min\": 0.40707964601769914,\n
\"max\": 0.8273809523809523,\n        \"num_unique_values\": 3,\n
\"samples\": [\n          0.7674418604651163,\n
0.40707964601769914,\n          0.8273809523809523\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Recall\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\":
0.24428924626498702,\n        \"min\": 0.3865546218487395,\n
\"max\": 0.8398791540785498,\n        \"num_unique_values\": 3,\n
```


Interpretation: Precision, recall, and F1 are consistent with the classification report: the Enrolled class has substantially lower scores. Future model comparisons should prioritize macro-F1 and per-class recall, not accuracy alone.

7. Seven Stages of Visualization

- Acquire — download the UCI dataset.
- Parse — inspect fields, types, and target labels.
- Filter — check and remove exact duplicate records; select predictor and target fields.
- Mine — calculate class counts, descriptive summaries, distributions, and correlations.

• Represent — choose class counts, histograms, a heatmap, boxplots, and evaluation charts.

• Refine — label axes, titles, scales, and legends; improve charts where coded categories are better shown as counts or proportions.

• Interact — inspect notebook tables and rerun cells for feature-level exploration; interactive charts are a future extension.

8. Conclusion and Future Scope

The analysis examined 4,424 student records across 37 fields and identified a class imbalance, with Graduate the largest class and Enrolled the smallest. The MLP achieved 73.64% test accuracy, but performance varied by class: the model identified Graduate and Dropout more reliably than Enrolled. The result demonstrates the value of evaluating per-class precision, recall, F1-score, and ROC-AUC alongside accuracy. Key limitations include the use of discrete-coded variables as numeric features, a basic two-configuration model comparison, and the weaker performance on the Enrolled class.

Future scope

• Compare MLP with decision trees, random forests, and gradient-boosting baselines.

• Use cross-validation and systematic hyperparameter tuning; consider a larger iteration budget.

• Improve visualization of coded categorical variables using frequency and proportion charts.

• Evaluate class weighting or resampling carefully to improve Enrolled recall.

• Validate on a newer cohort and review fairness implications before any real-world use.

9. References

• Realinho, V., Machado, J., Baptista, L., & Martins, M. V. (2022). Predict students' dropout and academic success [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5MC89

• Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

• Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90–95.

• Waskom, M. L. (2021). Seaborn: Statistical data visualization. Journal of Open Source Software, 6(60), 3021.

• McKinney, W. (2010). Data structures for statistical computing in Python. Proceedings of the 9th Python in Science Conference, 56–61.

Appendix: Source Code

The complete Python source code is available in the executed notebook Student_Performance_EDA_Project.ipynb. The report uses outputs captured in the submitted PDF export. Submit the notebook, this report, README, and dataset link in the public GitHub repository.